

EE345 Mod3-Lec1: Gradient Descent for Convex Costs

References:

- [OM]: Chapter 3.3, 4, 5 [Optimization Methods]

Overview

1. Module 1: Distances and the Algorithms they Enable

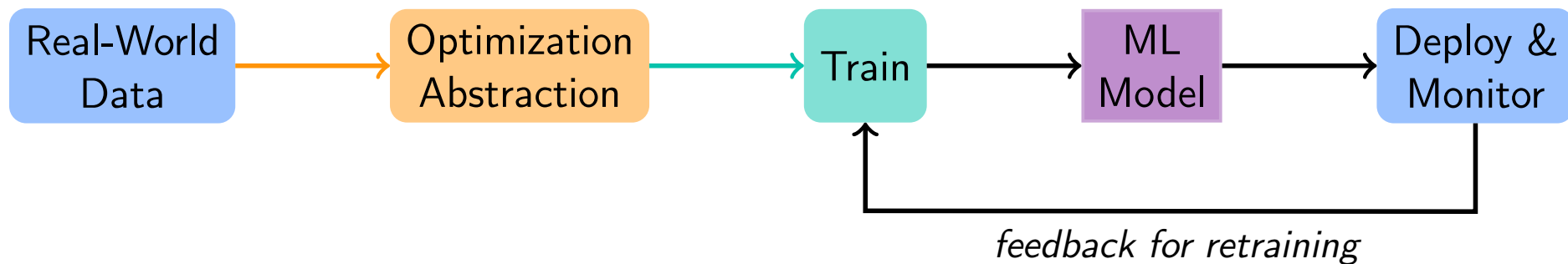
- ▶ Distances and metrics: Euclidean distance, Manhattan distance, cosine distance, worst-case distance, etc.
- ▶ Vectors and matrices: dot product, matrix multiplication, transpose, inverse, etc.
- ▶ Distance based algorithms: k-nearest neighbors (supervised), k-means (unsupervised)

2. Module 2: Linear Models & Regression

- ▶ Convex costs: quadratic functions/forms
- ▶ Linear models and their prevalence in machine learning
- ▶ Least squares: quadratic cost, linear regression

3. Module 3: Gradient Descent, Regularization, Kernel Methods

ML workflow: Role of Linear Algebra & Optimization



This course so far:

- **Optimization Abstraction**: Data $\rightarrow$ Optimization Abstraction via losses as norms
- **ML models**: Distance metric based models (k-NN, k-means); Linear models (least squares regression, attention, PCA)
- **Training**: Optimizing "convex" objectives (least squares)

Coming up next:

- **Training**: algorithms such as stochastic gradient descent (bias-variance tradeoff)
- **ML models**: Linear models: kernel methods
- **Optimization Abstraction**: Regularization, generalization, overfitting, etc.

ML workflow: Optimization Framework for Learning

Real world task involving data $\longrightarrow$ Optimization Problem $\longrightarrow$ ML model

Canonical Optimization Problem

$$\min_{\theta \in \Theta \subset \mathbb{R}^n} \ell(\theta)$$

Terminology:

- θ : Parameter/optimization or decision variable
- $\ell(\theta)$: Objective/Loss function; in ML will often depend on data Z ; write as $\ell(\theta; Z)$.
- $\Theta \subseteq \mathbb{R}^n$: Constraint set

How do we find solutions to optimization problems?

Last Time: From Stability to Regularization

- The minimum-norm solution is the **most stable** interpolant:

$$\rightarrow \theta^* = \arg \min_{\theta} \|\theta\|_2^2 \quad \text{s.t.} \quad X\theta = y.$$

$$\mathcal{J}(\theta; \mathcal{D}) = \|\theta\|^2 + \mathcal{D}^T (X\theta - y)$$

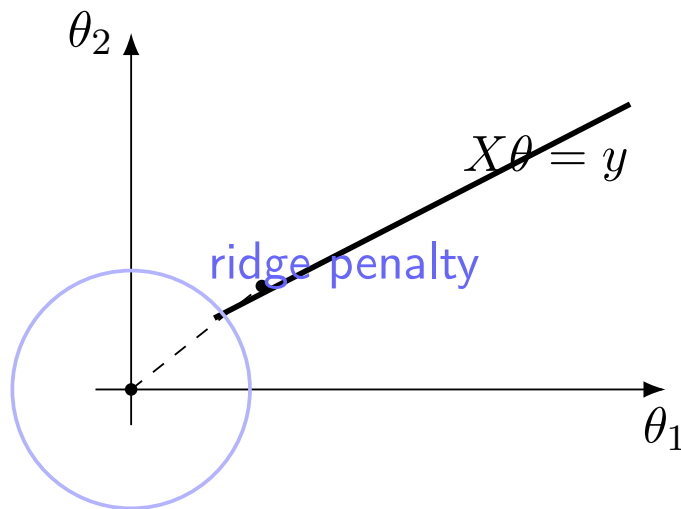
- In practice, data are noisy — exact interpolation can overfit.
- Idea: **trade off fit and simplicity.**

$$\min_{\theta} \|X\theta - y\|_2^2 + \lambda \|\theta\|_2^2$$

- $\lambda > 0$ controls the tradeoff.
- As $\lambda \rightarrow 0$, we recover the minimum-norm interpolant.
- As $\lambda \rightarrow \infty$, we prioritize simplicity ($\theta \approx 0$).

Last Time: Regularization = Controlled Stability

- Regularization keeps the solution **stable** when y changes slightly.
- It also reduces variance and improves prediction quality.
- Gradient descent with small initialization often behaves as if λ is small but positive (implicit regularization).



Geometrically: regularization picks the point on the affine set $X\theta \approx y$ closest to the smallest-norm ball.

From Least Squares to Optimization

- We solved

$$\min_{\theta} \frac{1}{2} \|y - X\theta\|_2^2.$$

- Closed-form solution: $\hat{\theta} = (X^\top X)^{-1} X^\top y$. $\leftarrow$
- But for **large datasets**, computing the inverse is **expensive** or **impossible**!
- Idea: Iteratively improve θ instead of solving exactly.

Optimization viewpoint

Find θ^* that minimizes a convex cost function $f(\theta)$.

Gradients and the Direction of Steepest Descent

$$\min_{\theta} f(\theta)$$

$\nabla f(\theta) = 0$

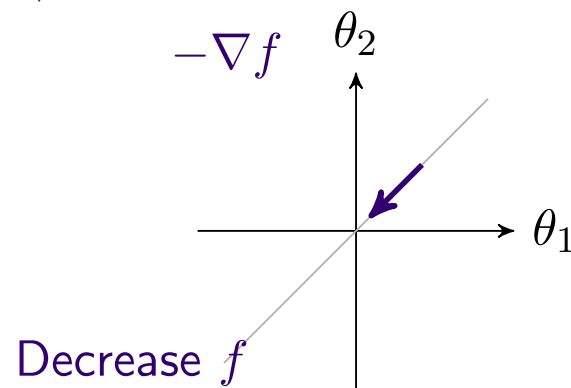
$$\nabla f(\theta) = 0$$

$$\nabla_{\theta} f(\theta) = \begin{bmatrix} \frac{\partial f}{\partial \theta_1} \\ \vdots \\ \frac{\partial f}{\partial \theta_d} \end{bmatrix}$$

$$\|X\theta - y\|^2$$

$$\theta_{k+1} = \theta_k - \eta \nabla f(\theta_k)$$

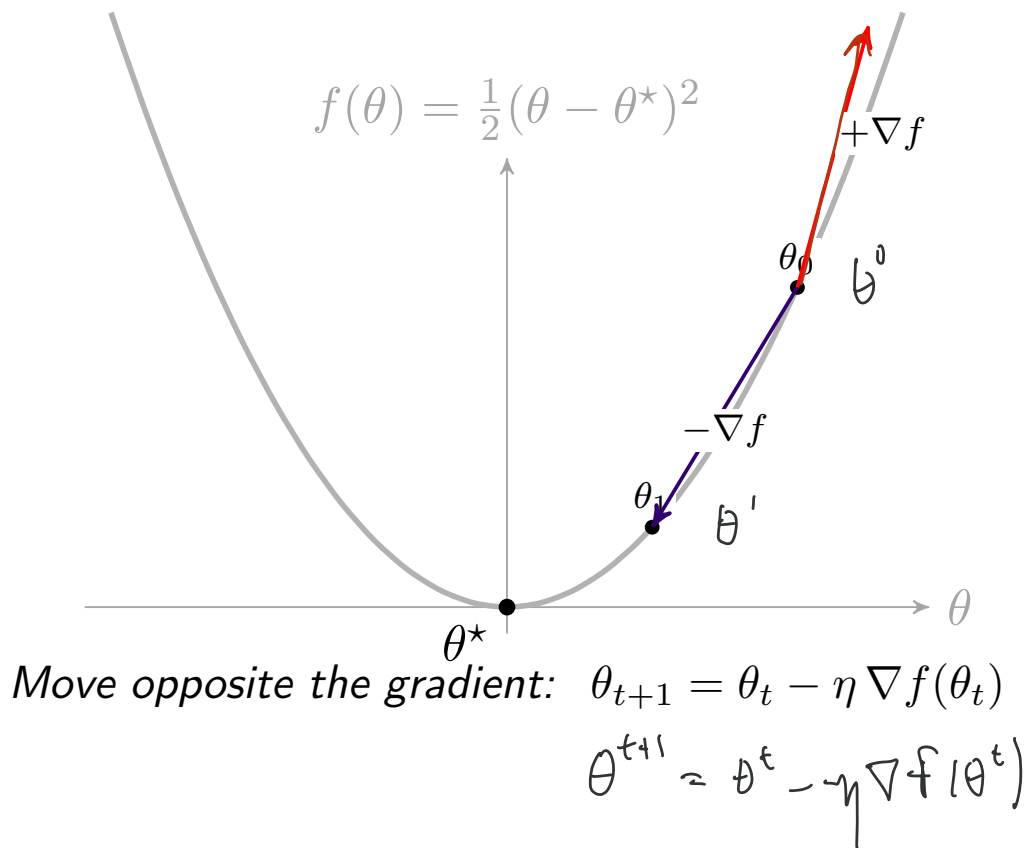
- Points in direction of fastest increase of f .
- So to decrease f , move *opposite* the gradient.



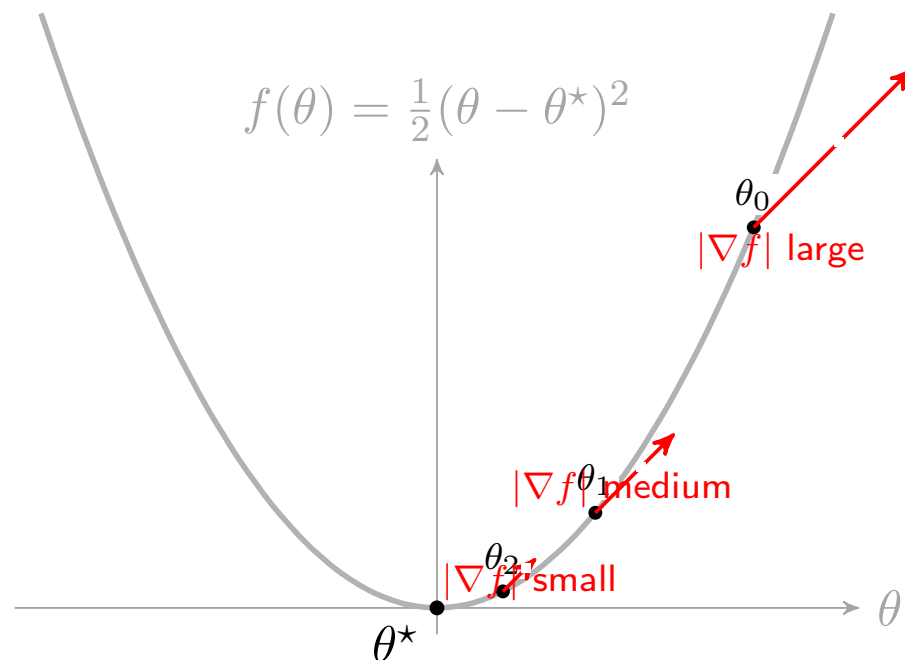
Gradient Descent: Clean Bowl Picture

$$\theta = \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}$$

$$\theta_t$$



Gradient Magnitude and Step Size



The gradient points uphill and its magnitude shrinks near θ^ , so gradient descent takes smaller steps.*

Lab 4

Example: Gradient Descent on a Quadratic

$$L(\theta) = \|X\theta^t - y\|^2 + \lambda R(\theta^t) \quad \leftarrow \text{loss fn.}$$

$$e(\theta^t) = \|\theta^t - \theta^*\|^2 + \lambda R(\theta^t) \quad \leftarrow \text{iterate converges}$$

tracking error

$$f(\theta) = \frac{1}{2}a\theta^2 - b\theta + c, \quad \nabla f(\theta) = a\theta - b.$$

$$a\theta = b$$

$$\Rightarrow \theta^* = \frac{b}{a}$$

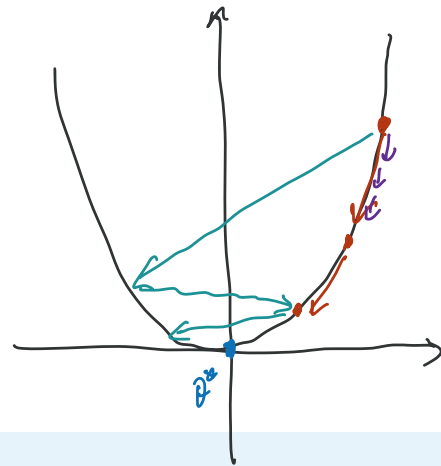
$$\Rightarrow \theta^{t+1} = \theta^t - \eta \nabla f(\theta^t) = \theta^t - \eta (a\theta^t - b)$$

- Converges $\theta^* = \frac{b}{a}$ if step-size $\eta \in (0, \frac{2}{a})$

- Step-size matters

η too small $\Rightarrow$ slow convergence

η too big $\Rightarrow$ oscillation



Key takeaway

Gradient descent finds the minimum of convex functions by iteratively moving opposite the gradient.

Gradient Descent Step Size: Quadratic Example

$$\| \theta^{t+1} - \theta^* \| = \| \theta^t - \eta (a \theta^t - b) - \theta^* \|$$

$$\stackrel{*}{=} \| \theta^t - \eta (a \theta^t - b) \pm (a \theta^* - b) - \theta^* \|$$

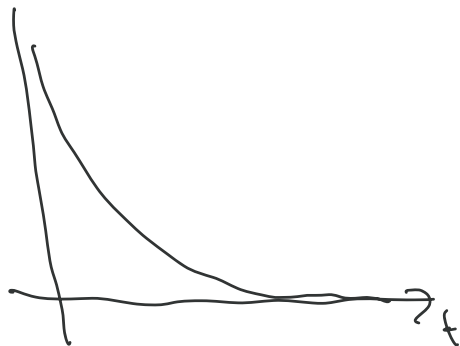
$$\leq \| 1 - \eta a \| \| \theta^t - \theta^* \| \quad (*)$$

$$\begin{aligned} \nabla f(\theta^*) &= 0 \\ &= a \theta^* - b \end{aligned}$$

1 - one step contraction

(*) will contract if

$$\| 1 - \eta a \| < 1$$



$$\| \theta^t - \theta^* \| \leq \underbrace{(1 - \eta a)^t}_{\leq e^{-\eta a t}} \| \theta^0 - \theta^* \|$$

$$\leq e^{-\eta a t} \| \theta^0 - \theta^* \|$$

$$\leq \varepsilon$$

$$t \geq \Omega\left(\log\left(\frac{R}{\varepsilon}\right)\right)$$

$$\log(e^{-\eta a t} \| \theta^0 - \theta^* \|) \leq \log \varepsilon$$

$$-\eta a t \leq \log\left(\frac{\varepsilon}{\| \theta^0 - \theta^* \|}\right)$$

Dynamical Systems View

$$\theta^T A \theta - b^T \theta + c$$

Gradient descent on a quadratic in d dimensions:

$$\| \theta^{t+1} - \theta^* \| \leq \| \underbrace{I - \eta A}_{\text{update matrix}} \| \| \theta^t - \theta^* \| \quad (\star)$$

$$x_{k+1} = Ax_k + Bu_k$$

Spectral radius:

$$\rho(M) := \max_i |\lambda_i(M)| \quad (\text{matrix input only}).$$

Convergence factor (as function of step size): Stability: $r(\eta) < 1$

$$r(\eta) := \max_i |1 - \eta \lambda_i(A)| < 1 \quad \text{for } \star \text{ to contract.}$$

$$\text{we need } \{ |1 - \eta \lambda_1(A)|, |1 - \eta \lambda_2(A)|, \dots, |1 - \eta \lambda_d(A)| \} < 1$$

In words: GD is a linear discrete-time system with update matrix $I - \eta A$. Its eigenvalues must lie inside the unit circle for stability.

Bounding the Error via Matrix Norms

For symmetric A , spectral radius is the largest eigenvalue of $I - \eta A$ in magnitude

$$\|I - \eta A\|_2 = \rho(I - \eta A) = \max_i |1 - \eta \lambda_i(A)|.$$

Hence

Since A is symmetric, eigenvalues describe both the norm and the stability.

Gradient Descent Dynamics in Least Squares

- Objective:

$$\eta \left(\frac{\partial}{\partial \theta} \left(\frac{1}{2} \|X\theta - y\|^2 \right) \right) \rightarrow a\theta^2 + b\theta + c$$

$$\theta^{t+1} = \theta^t - \eta \nabla f(\theta^t)$$

$$f(\theta) = \frac{1}{2} \|X\theta - y\|^2, \quad \nabla f(\theta) = \underbrace{X^\top (X\theta - y)}_{\text{gradient}}$$

$$\lambda, R(\theta), \eta$$

$$\theta^{t+1} = (I - \eta X^\top X) \theta^t + \eta X^\top y \quad \Leftarrow$$

$$\underbrace{(I - \eta X^\top X)}_{A(\eta)} \Rightarrow A(\eta) := I - \eta X^\top X$$

$$\theta^t = A(\eta) \left(A(\eta) \theta^{t-2} + \eta X^\top y \right) + \eta X^\top y = \eta \sum_{k=0}^{t-1} \underbrace{(I - \eta X^\top X)^k}_{(1-\eta a)^k} X^\top y$$

$$0 < \eta < \frac{2}{\|X^\top X\|} \quad \Leftarrow$$

Gradient Descent Dynamics in Least Squares

Unrolling the Gradient Descent Recursion (back-up slide)

- Gradient descent update:

$$\theta_{t+1} = (I - \eta X^\top X)\theta_t + \eta X^\top y, \quad \theta_0 = 0.$$

- Define

$$A := I - \eta X^\top X, \quad b := \eta X^\top y.$$

- Then $\theta_{t+1} = A\theta_t + b$.

Iterate (unroll) the recursion

$$\theta_1 = b,$$

$$\theta_2 = Ab + b,$$

$$\theta_3 = A^2b + Ab + b,$$

$$\vdots$$

so that

$$\theta_t = \sum_{k=0}^{t-1} A^k b.$$

Unrolling the Gradient Descent Recursion (back-up slide)

Substitute back:

$$\theta_t = \eta \sum_{k=0}^{t-1} (I - \eta X^\top X)^k X^\top y.$$

- This is a matrix geometric series.
- Recall standard geometric series:

$$1 + r + \dots + r^{t-1} = \frac{1 - r^t}{1 - r}.$$

Using the Matrix Geometric Series

$$\theta_t = \eta \sum_{k=0}^{t-1} (I - \eta X^\top X)^k X^\top y.$$

- Let $B = X^\top X$. The finite geometric series gives

$$\sum_{k=0}^{t-1} (I - \eta B)^k = (\eta B)^{-1} [I - (I - \eta B)^t].$$

- Substituting:

$$\boxed{\theta_t = (X^\top X)^{-1} [I - (I - \eta X^\top X)^t] X^\top y.}$$

The tricky part

Finite Steps $\implies$ Implicit Regularization

- That first identity gave us

$$\theta_t = (X^\top X)^{-1} [I - (I - \eta X^\top X)^t] X^\top y = \eta t B B^{-1} (I + \eta t B)^{-1} X^\top y = \eta t (I + \eta t B)^{-1} X^\top y$$

- Then the tricky part gives us this approximation:

$$\theta_t \approx \left(X^\top X + \frac{1}{\eta t} I \right)^{-1} X^\top y.$$

- This has the same form as **ridge regression**:

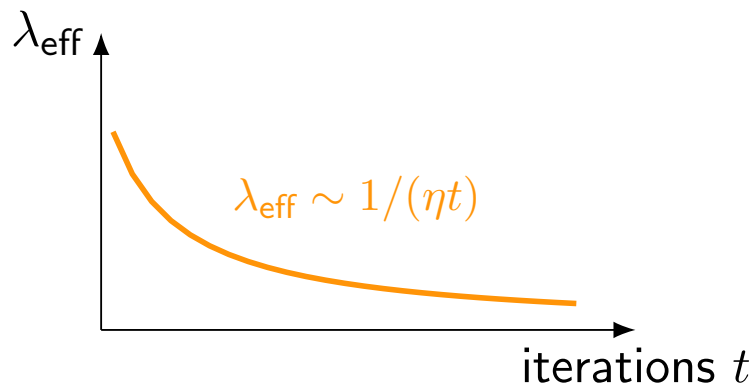
$$\theta_\lambda = (X^\top X + \lambda I)^{-1} X^\top y, \quad \text{with } \lambda_{\text{eff}} \approx \frac{1}{\eta t}.$$

Interpretation: “Implicit Regularization”

- Even though we did *not* add an explicit penalty, gradient descent acts as if there were a small ridge term.
- **Early stopping** corresponds to a larger effective λ : smoother, smaller-norm solution.
- **Running to convergence** corresponds to $\lambda_{\text{eff}} \rightarrow 0$: recovers the minimum-norm interpolant.

Gradient descent with small initialization behaves like ridge regression with $\lambda_{\text{eff}} \approx \frac{1}{\eta t}$.

Stopping gradient descent early acts like adding an ℓ_2 penalty: controls model complexity without explicit regularization.



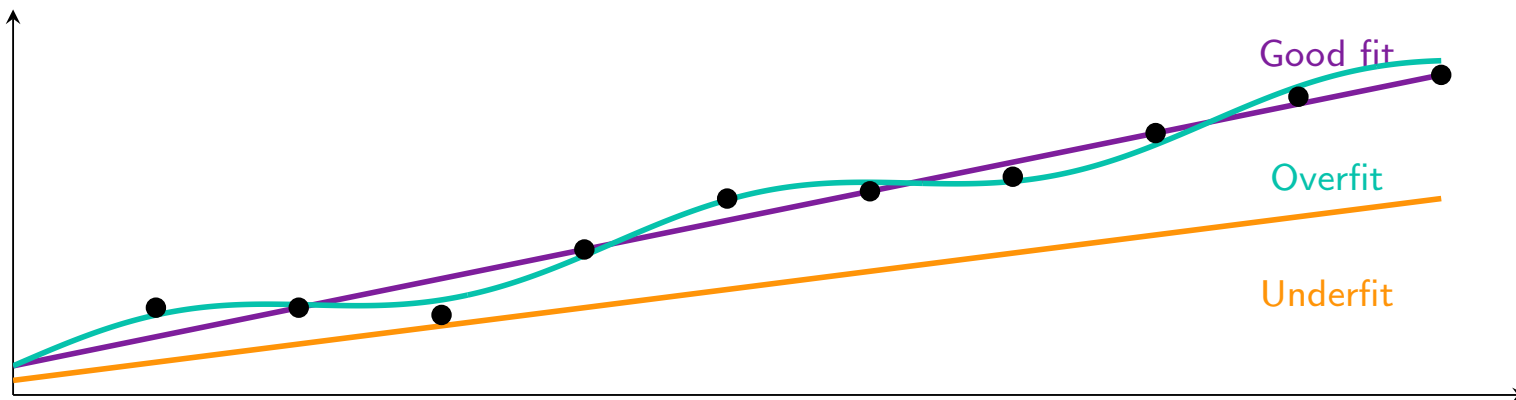
more training $\implies$ less regularization

Why Regularization?

- Our goal is to make predictions that work **beyond the training data**.
- If the model fits the training data too closely, it may fail on new examples — this is called **overfitting**.
- If the model is too simple, it cannot capture patterns — this is **underfitting**.
- Regularization helps us find a **middle ground**.

Underfitting vs Overfitting

- Same training data, three different models:
 - ▶ **Underfit (Tangerine)**: too simple, misses trend.
 - ▶ **Good fit (Purple)**: matches overall pattern.
 - ▶ **Overfit (Turquoise)**: too wiggly, chases every point.



Explicit Regularization

- We can directly control how large the parameter vector is.
- Example: **ridge regression**

$$\min_{\theta} \|X\theta - y\|^2 + \lambda\|\theta\|^2$$

- The second term keeps θ small.
 λ is the **regularization strength**.
 - ▶ λ small $\implies$ flexible model.
 - ▶ λ large $\implies$ smoother, simpler model.
- Same idea appears with ℓ_1 (lasso) and others. You just get different shapes for the regularization term.

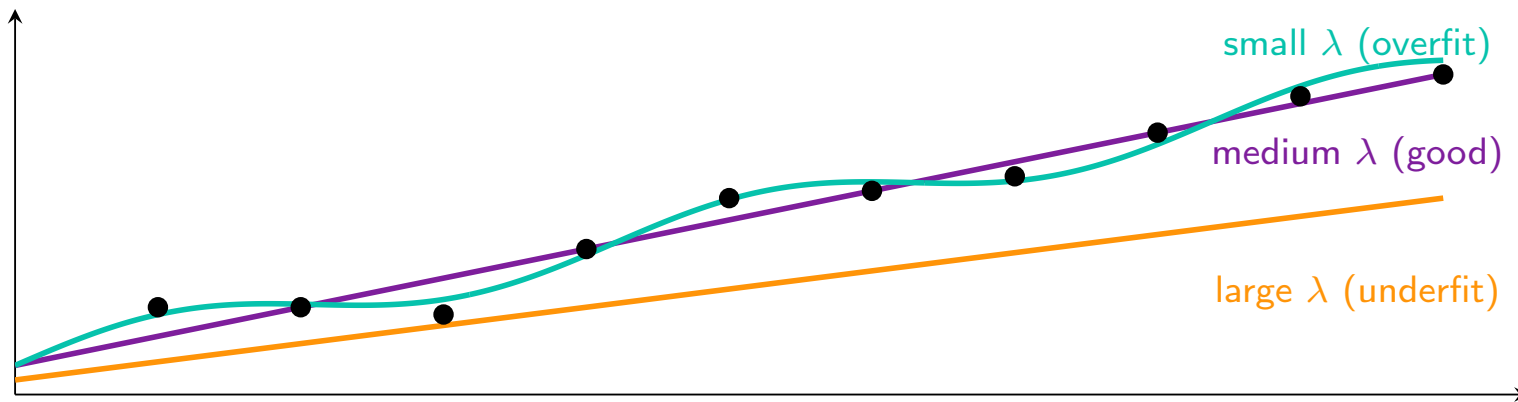
Explicit Regularization (Ridge): Effect of λ

- Ridge regression:

$$\min_{\theta} \|X\theta - y\|^2 + \lambda\|\theta\|^2$$

- λ controls model flexibility:

- ▶ **Large** λ : very simple $\implies$ underfit.
- ▶ **Medium** λ : good balance.
- ▶ **Small** λ : very flexible $\implies$ overfit.

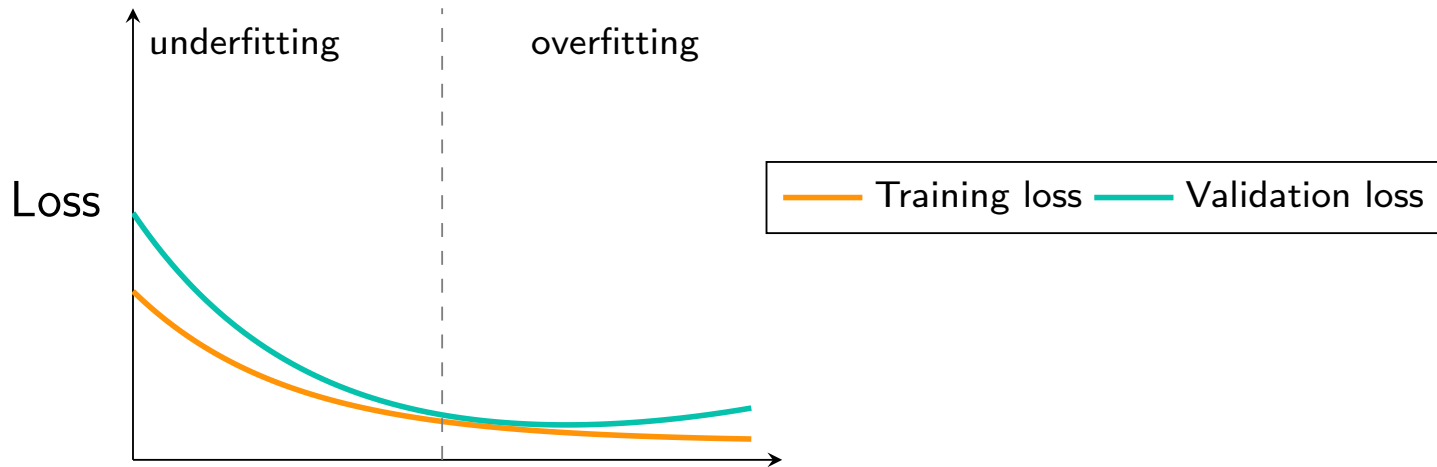


Implicit Regularization: From the Algorithm Itself

- Sometimes we do **not** add any penalty, but the training process still prefers “simpler” models.
- Examples:
 - ▶ Gradient descent on $\min_{\theta} \|X\theta - y\|^2$ (when there are many solutions) picks the one with the **smallest norm**.
 - ▶ Stopping gradient descent **early** often gives smoother, more stable predictions.
- These effects come from how the algorithm moves and stops, not from an explicit penalty term.

Effect of Regularization on Training and Validation Loss

- Strong regularization (λ large): model too simple $\Rightarrow$ underfitting.
- Weak regularization (λ small): model too flexible $\Rightarrow$ overfitting.
- The sweet spot minimizes validation loss.



Regularization strength λ ($\rightarrow$ smaller = more complex model)

Early Stopping Acts Like Regularization

- Each gradient step changes θ by $-\eta \nabla_{\theta} \|X\theta - y\|^2$.
- If we stop after only a few steps, θ is still close to 0.
- This behaves like shrinking θ with an ℓ_2 penalty.
- So early stopping is a kind of **implicit regularization**.
- In practice:
 - ▶ Track training and validation loss.
 - ▶ Stop when validation loss starts increasing.

Early Stopping as Implicit Regularization

- As we train (steps $t = 0, 1, 2, \dots$):
 - ▶ **Training loss** keeps decreasing.
 - ▶ **Validation loss** decreases at first, then increases once we overfit.
- Stop at t^* where validation loss is smallest — this is **early stopping**.
- Effect: prevents overly complex parameters $\implies$ behaves like regularization.

